

# 漢堡王 ERP — dev 運行環境部署參考手冊

參考環境：台南 61.218.209.214 連線帳密：burgerking / Ns22645531 postgres:账号/密碼：postgres / Newsoft\_0255

適用系統：Ubuntu 24.04.3 LTS

CPU 配置：4 核 4 线程（虚拟 CPU）

内存配置：总内存：6 GB

存储配置：总磁盘容量：500 GB （最小60GB）

系统环境：架构：x86\_64 VMware

## 目錄

- 第一部分：初次部署
  - [步驟 1：準備 Linux Ubuntu 24.04.3 LTS 環境]
  - [步驟 2：安裝 PostgreSQL 16.13]
  - [步驟 3：安裝 Redis 7.0.15]
  - [步驟 4：安裝 JDK 17 及 Maven 3.8.7]
  - [步驟 5：安裝 Nginx 1.24.0]
  - [步驟 6：安裝 Git 2.43.0]
  - [步驟 7：安裝 Node.js 21.7.3]
  - [步驟 8：拉取項目程式]
  - [步驟 9：調整後端配置文件並編譯]
  - [步驟 10：調整前端配置並編譯]
  - [步驟 11：設置 Nginx]
  - [步驟 12：設置後端服務開機啟動]
- 第二部分：今後部署（日常更新）
  - [1. 後端代碼更新部署]
  - [2. 前端代碼更新部署]

## 第一部分：初次部署

### 步驟 1：準備 Linux Ubuntu 24.04.3 LTS 環境

以 Ubuntu 24.04.3 LTS 為例，確保系統環境已就緒。

```
burgerking@burgerkingerpserver:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
burgerking@burgerkingerpserver:~$
```

## 步驟 2：安裝 PostgreSQL 16.13

### 2.1 更新系統軟件包

```
sudo apt update && sudo apt upgrade -y
```

### 2.2 安裝 PostgreSQL

```
sudo apt install postgresql postgresql-contrib -y
```

### 2.3 確認 PostgreSQL 啟動狀態

安裝完成後，PostgreSQL 會自動啟動。執行以下命令檢查狀態：

```
sudo systemctl status postgresql
```

```
burgerking@burgerkingerpserver:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2025-12-04 13:48:21 CST; 4 months 9 days ago
     Main PID: 407889 (code=exited, status=0/SUCCESS)
        CPU: 1ms

Notice: journal has been rotated since unit was started, output may be incomplete.
burgerking@burgerkingerpserver:~$
```

### 2.4 設置開機自啟

```
sudo systemctl enable postgresql
```

### 2.5 修改遠程訪問配置

編輯 `pg_hba.conf` 文件，允許遠程連接：

```
sudo vi /etc/postgresql/16/main/pg_hba.conf
```

```
GNU nano 7.2 /etc/postgresql/16/main/pg_hba.conf *
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local all postgres peer

# TYPE DATABASE USER ADDRESS METHOD

# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
host all all 127.0.0.1/32 scram-sha-256
# IPv6 local connections:
host all all ::1/128 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
host all all 0.0.0.0/0 md5

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo
```

在文件中加入以下一行並保存：

```
host all all 0.0.0.0/0 md5
```

## 2.6 修改監聽地址

編輯 `postgresql.conf` 文件：

```
sudo vi /etc/postgresql/16/main/postgresql.conf
```

找到 `listen_addresses` 並修改為：

```
listen_addresses = '*'
```

保存後，如下圖所示：

```
external_pid_file = '/var/run/postgresql/16-main.pid'           # write an extra PID file
                                                                # (change requires restart)

#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100           # (change requires restart)
#reserved_connections = 0       # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
                                # (change requires restart)
#unix_socket_group = ''        # (change requires restart)
#unix_socket_permissions = 0777 # begin with 0 to use octal notation

-- INSERT --
```

## 2.7 重啟 PostgreSQL

```
sudo systemctl restart postgresql
```

## 2.8 設定默認用戶密碼

登入默認用戶：

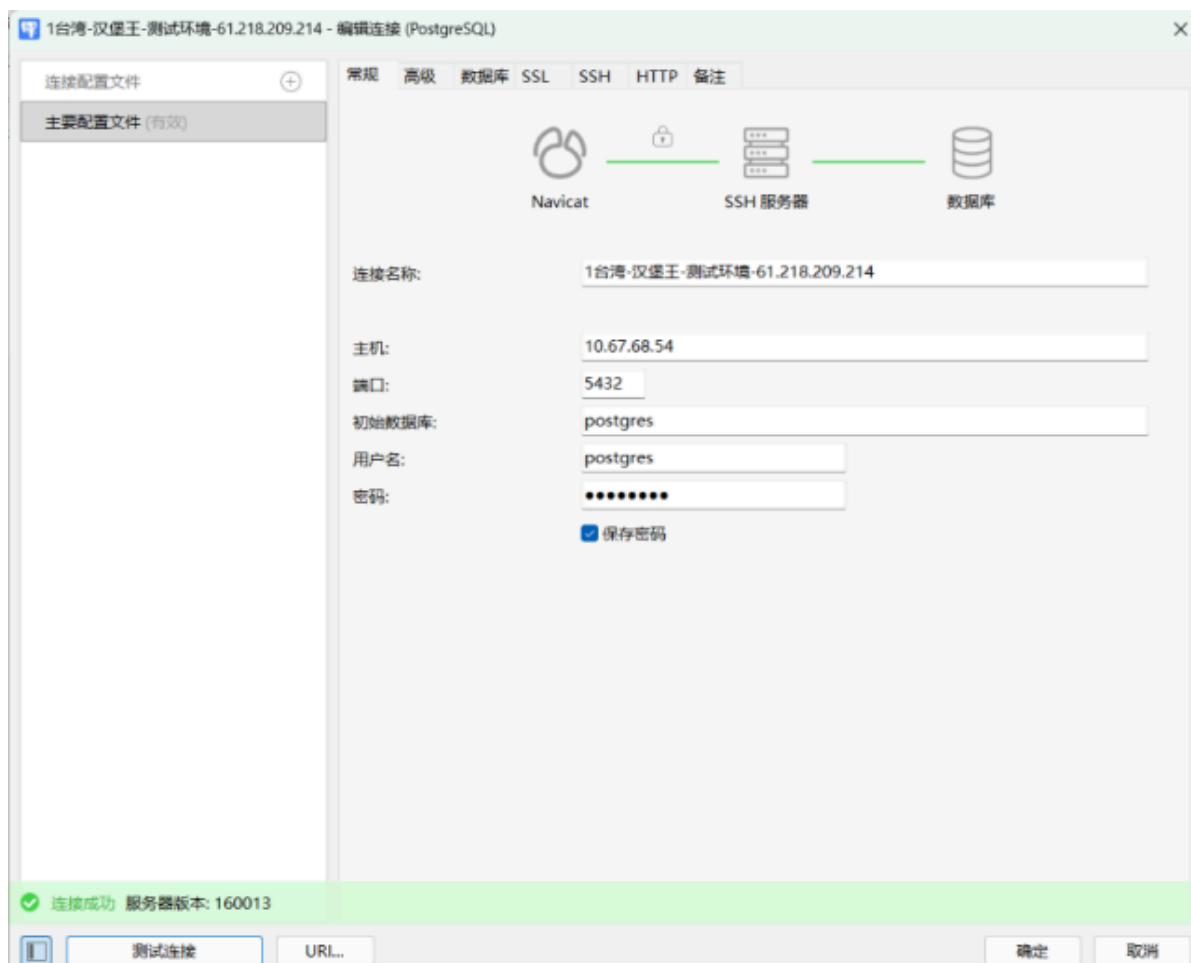
```
sudo -u postgres psql
```

設定密碼（以 Newsoft\_0255 為例）：

```
ALTER USER postgres WITH PASSWORD 'Newsoft_0255';
```

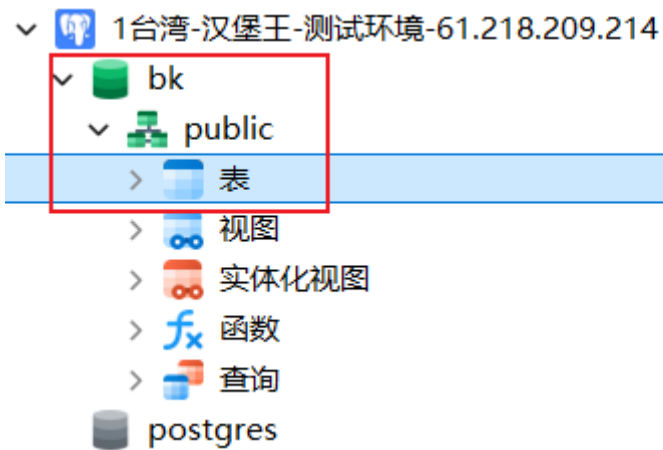
## 2.9 驗證遠程連接

使用數據庫管理工具（如 DBeaver、TablePlus）進行遠程訪問驗證：



## 2.10 創建業務數據庫

創建名為 bk 的數據庫，並將 61.218.209.214 環境下的 bk 數據庫導入：



## 步驟 3：安裝 Redis 7.0.15

### 3.1 安裝 Redis

```
sudo apt install redis-server -y
```

### 3.2 檢查安裝狀態

```
sudo systemctl status redis-server
```

### 3.3 設置開機自啟

```
sudo systemctl enable redis-server
```

### 3.4 修改 Redis 配置文件

```
sudo vi /etc/redis/redis.conf
```

按需修改以下配置項（如綁定地址、密碼等）：

```
# IF YOU ARE SURE YOU WANT YOUR INSTANCE TO LISTEN TO ALL THE INTERFACES
# COMMENT OUT THE FOLLOWING LINE.
#
# You will also need to set a password unless you explicitly disable protected
# mode.
# ~~~~~
bind 0.0.0.0 -:::1

# By default, outgoing connections (from replica to master, from Sentinel to
# instances, cluster bus, etc.) are not bound to a specific local address. In
# most cases, this means the operating system will handle that based on routing
# and the interface through which the connection goes out.
#
# Using bind-source-addr it is possible to configure a specific address to bind
# to, which may also affect how the connection gets routed.
#
# Example:
#
# bind-source-addr 10.0.0.1

# Protected mode is a layer of security protection, in order to avoid that
# Redis instances left open on the internet are accessed and exploited.
-- INSERT --
```

```
# Protected mode is a layer of security protection, in order to avoid that
# Redis instances left open on the internet are accessed and exploited.
#
# When protected mode is on and the default user has no password, the server
# only accepts local connections from the IPv4 address (127.0.0.1), IPv6 address
# (:::1) or Unix domain sockets.
#
# By default protected mode is enabled. You should disable it only if
# you are sure you want clients from other hosts to connect to Redis
# even if no authentication is configured.
protected-mode no

# Redis uses default hardened security configuration directives to reduce the
# attack surface on innocent users. Therefore, several sensitive configuration
# directives are immutable, and some potentially-dangerous commands are blocked.
#
# Configuration directives that control files that Redis writes to (e.g., 'dir'
# and 'dbfilename') and that aren't usually modified during runtime
# are protected by making them immutable.
#
# Commands that can increase the attack surface of Redis and that aren't usually
# called by users are blocked by default.
-- INSERT --
```

111,18

4%

### 3.5 重啟 Redis 服務

```
sudo systemctl restart redis-server
```

## 步驟 4：安裝 JDK 17 及 Maven

### 4.1 安裝 JDK 17

```
sudo apt install openjdk-17-jre-headless
```

驗證安裝：

```
java -version
```

```
burgerking@burgerkingerpserver:~$ java -version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
burgerking@burgerkingerpserver:~$
```

### 4.2 安裝 Maven 3.8.7

```
sudo apt install maven
```

驗證安裝：

```
mvn -v
```

```
burgerking@burgerkingerpserver:~$ mvn -v
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-87-generic", arch: "amd64", family: "unix"
burgerking@burgerkingerpserver:~$
```

---

## 步驟 5: 安裝 Nginx 1.24.0

```
sudo apt install nginx -y
```

查看啟動狀態:

```
sudo systemctl status nginx
```

設置開機自啟:

```
sudo systemctl enable nginx
```

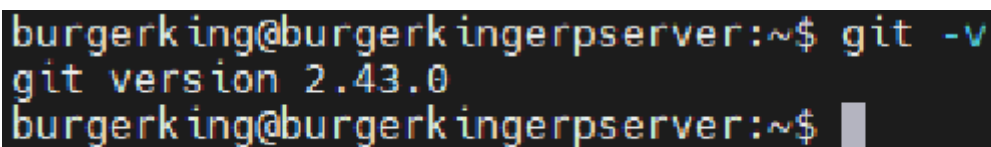
---

## 步驟 6: 安裝 Git 2.43.0

```
sudo apt install git -y
```

驗證安裝:

```
git -v
```

A terminal window with a black background and white text. The prompt is 'burgerking@burgerkingerpserver:~\$'. The user enters 'git -v' and the output is 'git version 2.43.0'. The prompt returns to 'burgerking@burgerkingerpserver:~\$' with a cursor.

```
burgerking@burgerkingerpserver:~$ git -v
git version 2.43.0
burgerking@burgerkingerpserver:~$
```

---

## 步驟 7: 安裝 Node.js 21.7.3

推薦使用 **nvm** 安裝 Node.js, 可靈活切換 Node 版本。

### 7.1 安裝 nvm

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

### 7.2 加載 nvm

```
source ~/.bashrc
```

### 7.3 安裝並啟用 Node 21

```
nvm install 21
nvm use 21
```

## 7.4 驗證 Node 21.7.3 及 npm 10.5.0

```
node -v  
npm -v
```

```
burgerking@burgerkingerpserver:~$ node -v  
v21.7.3  
burgerking@burgerkingerpserver:~$ npm -v  
10.5.0  
burgerking@burgerkingerpserver:~$
```

## 7.5 安裝 pnpm

```
npm install -g pnpm
```

驗證安裝：

```
pnpm -v
```

```
burgerking@burgerkingerpserver:~$ pnpm -v  
10.24.0  
burgerking@burgerkingerpserver:~$
```

# 步驟 8：拉取項目程式

## 8.1 创建工作目錄

```
sudo mkdir /work/  
sudo mkdir /work/Burgerking  
cd /work/Burgerking  
sudo chown -R $USER:$USER /work
```

## 8.2 克隆倉庫

提示：使用以下命令可記住 Git 密碼：

```
sudo git config --global credential.helper store
```

台灣環境可將 61.218.209.220 替換為 10.67.68.48，下同。

克隆後端項目：

```
# 西安環境  
git clone http://61.218.209.220/Burgerking/erp-spring.git  
  
# 台灣內網環境  
git clone http://10.67.68.48/Burgerking/erp-spring.git
```

克隆前端項目：

```
# 西安環境
git clone http://61.218.209.220/shinerice/erp-frontend.git

# 台灣內網環境
git clone http://10.67.68.48/shinerice/erp-frontend.git
```

## 步驟 9：調整後端配置文件並編譯

### 9.1 切換到後端目錄並選擇分支

```
cd /work/BurgerKing/erp-spring

# 該環境需要運行 dev-tw 分支所對應的程式
git checkout dev-tw
```

### 9.2 編輯應用配置文件

```
vi kingmaker-server/src/main/resources/application-dev.yml
```

進行關於數據庫、Redis 的設置修改並保存：

```
# 數據源配置項
datasource:
  druid: # Druid 【監控】相關的全局配置
    web-stat-filter:
      enabled: true
    stat-view-servlet:
      enabled: true
      allow: # 設置白名單，不填則允許所有訪問
      url-pattern: /druid/*
      login-username: # 控制台管理用戶名和密碼
      login-password:
    filter:
      stat:
        enabled: true
        log-slow-sql: true # 慢 SQL 記錄
        slow-sql-millis: 100
        merge-sql: true
      wall:
        config:
          multi-statement-allow: true
dynamic: # 多數據源配置
druid: # Druid 【連接池】相關的全局配置
  initial-size: 5 # 初始連接數
  min-idle: 10 # 最小連接池數量
  max-active: 20 # 最大連接池數量
  max-wait: 600000 # 配置獲取連接等待超時的時間，單位：毫秒
  time-between-connection-validation-interval: 60000 # 配置間隔多久才進行一次檢測，檢測需要關閉的空閒連接，單位：毫秒
  min-evictable-idle-time-millis: 300000 # 配置一個連接在池中最小生存的時間，單位：毫秒
  max-evictable-idle-time-millis: 900000 # 配置一個連接在池中最大生存的時間，單位：毫秒
  validation-query: SELECT 1 # 配置檢測連接是否有效
  test-while-idle: true
  test-on-borrow: false
  test-on-return: false
primary: master
datasource:
  master:
    url: jdbc:postgresql://127.0.0.1:5432/bk
    username: postgres
    password: Newsoft 0255
  slave: # 模擬從庫，可根據自己需要修改 # 模擬從庫，可根據自己需要修改
    lazy: true # 開啟懶加載，保證啟動速度
    url: jdbc:mysql://127.0.0.1:3306/ruoyi-vue-pro?useSSL=false&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true&nullCatalogMeansCurrent=
connector/J 8.X 連接的示例
#
username: root
password: 123456

# Redis 配置。Redisson 默認的配置足夠使用，一般不需要進行調優
```

### 9.3 執行編譯

```
mvn clean package -Dmaven.test.skip=true
```

### 9.4 創建部署腳本

```
touch deploy.sh
vi deploy.sh
```

將以下內容填入並保存：

```
#!/bin/bash
set -e

DATE=$(date +%Y%m%d%H%M)

# 基础路径
BASE_PATH=/work/BurgerKing/erp-spring/kingmaker-server/target/

# 服务名称。同时约定部署服务的 jar 包名字也为它。
SERVER_NAME=kingmaker-server

# 环境
PROFILES_ACTIVE=dev

# heapError 存放路径
HEAP_ERROR_PATH=$BASE_PATH/heapError

# JVM 参数
JAVA_OPS="-Xms512m -Xmx512m -XX:+HeapDumpOnOutOfMemoryError -
XX:HeapDumpPath=$HEAP_ERROR_PATH"

# Skywalking Agent 配置（如需启用，取消以下注释）
#export SW_AGENT_NAME=$SERVER_NAME
#export SW_AGENT_COLLECTOR_BACKEND_SERVICES=192.168.0.84:11800
#export SW_GRPC_LOG_SERVER_HOST=192.168.0.84
#export SW_AGENT_TRACE_IGNORE_PATH="Redisson/PING,/actuator/**,/admin/**"
#export JAVA_AGENT=-javaagent:/work/skywalking/apache-skywalking-apm-
bin/agent/skywalking-agent.jar

# 停止：优雅关闭之前已经启动的服务
function stop() {
    echo "[stop] 开始停止 $BASE_PATH/$SERVER_NAME"
    PID=$(ps -ef | grep $BASE_PATH/$SERVER_NAME | grep -v "grep" | awk '{print $2}')
    if [ -n "$PID" ]; then
        echo "[stop] $BASE_PATH/$SERVER_NAME 运行中，开始 kill [$PID]"
        kill -15 $PID
        for ((i = 0; i < 120; i++))
        do
            sleep 1
            PID=$(ps -ef | grep $BASE_PATH/$SERVER_NAME | grep -v "grep" |
awk '{print $2}')
            if [ -n "$PID" ]; then
                echo -e ".\c"
            else
                echo '[stop] 停止 $BASE_PATH/$SERVER_NAME 成功'
                break
            fi
        done
        if [ -n "$PID" ]; then
            echo "[stop] $BASE_PATH/$SERVER_NAME 失败，强制 kill -9 $PID"
            kill -9 $PID
        fi
    else
        echo "[stop] $BASE_PATH/$SERVER_NAME 未启动，无需停止"
```

```

    fi
}

# 启动：启动后端项目
function start() {
    echo "[start] 开始启动 $BASE_PATH/$SERVER_NAME"
    echo "[start] JAVA_OPS: $JAVA_OPS"
    echo "[start] JAVA_AGENT: $JAVA_AGENT"
    echo "[start] PROFILES: $PROFILES_ACTIVE"
    nohup java -server $JAVA_OPS $JAVA_AGENT -jar $BASE_PATH/$SERVER_NAME.jar --
spring.profiles.active=$PROFILES_ACTIVE > nohup.out 2>&1 &
    echo "[start] 启动 $BASE_PATH/$SERVER_NAME 完成"
}

# 部署
function deploy() {
    cd $BASE_PATH
    stop
    start
}

deploy

```

## 9.5 啟動後端服務

```
bash deploy.sh
```

```

rice@rice:/work/ShineRice/erp-backend$ bash deploy.sh
[stop] 开始停止 /work/ShineRice/erp-backend/kingmaker-server/target//kingmaker-server
[stop] /work/ShineRice/erp-backend/kingmaker-server/target//kingmaker-server 未启动, 无需停止
[start] 开始启动 /work/ShineRice/erp-backend/kingmaker-server/target//kingmaker-server
[start] JAVA_OPS: -Xms512m -Xmx512m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/work/ShineRice/erp-backend/kingmaker-server/target//heapError
[start] JAVA_AGENT:
[start] PROFILES: dev
[start] 启动 /work/ShineRice/erp-backend/kingmaker-server/target//kingmaker-server 完成
rice@rice:/work/ShineRice/erp-backend$

```

## 9.6 查看後端日誌

```
tail -500f ~/logs/kingmaker-server.log
```

## 步驟 10：調整前端配置並編譯

### 10.1 切換到前端目錄並選擇分支

```

cd /work/BurgerKing/erp-frontend

# 該環境需要運行 xian 分支所對應的程式
git checkout xian

```

### 10.2 修改環境配置文件

```
vi .env.dev
```

修改請求地址（指向後端服務）：

```
# 开发环境：本地只启动前端项目，依赖开发环境（后端、APP）
NODE_ENV=production

VITE_DEV=true

# 请求路径
VITE_BASE_URL='http://192.168.29.172:48080'

# 文件上传类型：server - 后端上传， client - 前端直连上传，仅支持S3服务
VITE_UPLOAD_TYPE=server

# 接口地址
VITE_API_URL=/admin-api

# 是否删除debugger
VITE_DROP_DEBUGGER=false

# 是否删除console.log
VITE_DROP_CONSOLE=false

# 是否sourcemap
VITE_SOURCEMAP=true

# 打包路径
VITE_BASE_PATH=/
```

### 10.3 設置 Node.js 内存上限

```
export NODE_OPTIONS="--max-old-space-size=2548"
```

將此設置永久生效（加入 `~/.bashrc`）：

```
vi ~/.bashrc
# 在文件末尾添加：
export NODE_OPTIONS="--max-old-space-size=2548"
```

保存後執行 `source ~/.bashrc` 使其立即生效。

### 10.4 安裝依賴並編譯

安裝依賴：

```
pnpm install
```

執行編譯：

```
npm run build:dev
```

編譯完成後如下圖所示：

```
rice@rice:/work/ShineRice/erp-frontend$ cd dist
rice@rice:/work/ShineRice/erp-frontend/dist$ ls
assets  favicon.ico  home.png  index.html  kingmaker-logo.png  kingmaker-logo-w.png  logo.gif
rice@rice:/work/ShineRice/erp-frontend/dist$
```

## 步驟 11: 設置 Nginx

### 11.1 編輯 Nginx 配置

```
sudo vi /etc/nginx/nginx.conf
```

編輯為以下內容:

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;

    gzip on;
    gzip_min_length 1k;      # 设置允许压缩的页面最小字节数
    gzip_buffers 4 16k;      # 用来存储 gzip 的压缩结果
    gzip_http_version 1.1;   # 识别 HTTP 协议版本
    gzip_comp_level 2;       # 设置 gzip 的压缩比 1-9 (1 最快, 9 压缩比最高)
    gzip_types text/plain text/css application/json application/javascript
    text/xml application/xml application/xml+rss text/javascript;
    gzip_proxied any;        # 无论后端 headers 返回什么, 均启用压缩

    server {
        listen 80;
        server_name 10.67.68.54; ## 重要!!! 修改成你的外网 IP/域名

        location / { ## 前端项目
            root /work/BurgerKing/erp-frontend/apps/burger-king/dist;
            index index.html index.htm;
            try_files $uri $uri/ /index.html;
        }

        location /admin-api/ { ## 后端项目 - 管理后台
            proxy_pass http://10.67.68.54:48080/admin-api/; ## 重要!!! 需修改为后
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header REMOTE-HOST $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        location /app-api/ { ## 后端项目 - 用户 App
```

端服务器 IP

```
proxy_pass http://10.67.68.54:48080/app-api/; ## 重要!!! 需修改为后端
服务器 IP

proxy_set_header Host $http_host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header REMOTE-HOST $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

}

}
```

## 11.2 驗證 Nginx 配置語法

```
sudo nginx -t
```

```
burgerking@burgerkingerpserver:/work/BurgerKing/erp-frontend$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
burgerking@burgerkingerpserver:/work/BurgerKing/erp-frontend$
```

## 11.3 重啟 Nginx 使配置生效

```
sudo nginx -s reload
```

## 11.4 訪問驗證

部署完成後，訪問以下地址查看效果（對應映射的外網地址）：

```
http://61.218.209.214/
```

## 步驟 12：設置後端服務開機啟動

將啟動後端服務加入開機運行腳本：

```
crontab -e
```

添加以下行並保存：

```
@reboot /work/BurgerKing/erp-spring/deploy.sh
```

## 第二部分：今後部署（日常更新）

### 1. 後端代碼更新部署

#### 完整更新流程

```
cd /work/BurgerKing/erp-spring
git pull
mvn clean package -Dmaven.test.skip=true
bash deploy.sh
```

## 快速重啟後端（不重新編譯）

```
bash /work/BurgerKing/erp-spring/deploy.sh
```

## 查看後端控制台日誌

```
tail -500f ~/logs/kingmaker-server.log
```

## 2. 前端代碼更新部署

⚠ 注意：服務器上編譯需要至少 **6GB 內存**，否則可能編譯失敗（`dist` 目錄中缺少 `assets` 文件夾）。

### 方式一：使用開發模式啟動（pnpm dev）

```
cd /work/BurgerKing/erp-frontend
git pull
pnpm dev:bk
sudo nginx -s reload
```

輸入密碼：123456

出現以下提示，代表編譯成功：

```
The CJS build of Vite's Node API is deprecated. See https://vitejs.dev/guide/troubleshooting#vite-cjs-api-deprecated for more details.

Building [] 0% | Transforms: 2/4137 | Chunks: 0/384 | Time: 0.0sBrowserslist: browser
ld. Please run:
  npx update-browserslist-db@latest
  Why you should do it regularly: https://github.com/browserslist/update-db#readme
Building [] 100% | Transforms: 4137/4137 | Chunks: 384/384 | Time: 82.6s

Build successful. Please see dist directory
```

若出現 `Killed` 字樣，則代表編譯失敗：

```
No target browser changes
rice@rice:/work/ShineRice/erp-frontend$ npm run build:dev

> yudao-ui-admin-vue3@2.4.1-snapshot build:dev
> node --max_old_space_size=4096 ./node_modules/vite/bin/vite.js build --mode dev

The CJS build of Vite's Node API is deprecated. See https://vitejs.dev/guide/troubleshooting#vite-cjs-api-deprecated for more details.

Building [] 95% | Transforms: 4137/4137 | Chunks: 384/384 | Time: 33.5sKilled
rice@rice:/work/ShineRice/erp-frontend$ vi package.json
rice@rice:/work/ShineRice/erp-frontend$ npm run build:dev
```

### 方式二：使用生產模式編譯（pnpm build）

```
cd /work/BurgerKing/erp-frontend
git pull
cd apps/burger-king
pnpm build:bk
```

### 3.怎么确定系统配置成功

在另外一台电脑访问 <http://61.218.209.214/auth/login> 连接成功出现下面的界面，表示配置成功



### 附錄

#### 常用服務管理命令

| 服務         | 啟動                                                      | 停止                                            | 重啟                                               | 狀態查看                                                |
|------------|---------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|-----------------------------------------------------|
| PostgreSQL | <code>sudo systemctl start postgresql</code>            | <code>sudo systemctl stop postgresql</code>   | <code>sudo systemctl restart postgresql</code>   | <code>sudo systemctl status postgresql</code>       |
| Redis      | <code>sudo systemctl start redis-server</code>          | <code>sudo systemctl stop redis-server</code> | <code>sudo systemctl restart redis-server</code> | <code>sudo systemctl status redis-server</code>     |
| Nginx      | <code>sudo systemctl start nginx</code>                 | <code>sudo systemctl stop nginx</code>        | <code>sudo nginx -s reload</code>                | <code>sudo systemctl status nginx</code>            |
| 後端應用       | <code>bash /work/BurgerKing/erp-spring/deploy.sh</code> | 見 deploy.sh stop()                            | 見 deploy.sh deploy()                             | <code>tail -500f ~/logs/kingmaker-server.log</code> |

#### 重要路徑匯總

| 說明      | 路徑                                       |
|---------|------------------------------------------|
| 後端項目根目錄 | <code>/work/BurgerKing/erp-spring</code> |

| 說明                | 路徑                                                                                   |
|-------------------|--------------------------------------------------------------------------------------|
| 前端項目根目錄           | /work/BurgerKing/erp-frontend                                                        |
| 後端配置文件            | /work/BurgerKing/erp-spring/kingmaker-server/src/main/resources/application-dev.yaml |
| 前端環境配置            | /work/BurgerKing/erp-frontend/.env.dev                                               |
| 前端編譯輸出            | /work/BurgerKing/erp-frontend/apps/burger-king/dist                                  |
| 部署腳本              | /work/BurgerKing/erp-spring/deploy.sh                                                |
| Nginx 配置          | /etc/nginx/nginx.conf                                                                |
| PostgreSQL HBA 配置 | /etc/postgresql/16/main/pg_hba.conf                                                  |
| Redis 配置          | /etc/redis/redis.conf                                                                |
| 後端運行日誌            | ~/logs/kingmaker-server.log                                                          |

## Git 倉庫地址

| 倉庫                | 西安地址                                             | 台灣內網地址                                        |
|-------------------|--------------------------------------------------|-----------------------------------------------|
| 後端 (erp-spring)   | http://61.218.209.220/BurgerKing/erp-spring.git  | http://10.67.68.48/BurgerKing/erp-spring.git  |
| 前端 (erp-frontend) | http://61.218.209.220/shinerice/erp-frontend.git | http://10.67.68.48/shinerice/erp-frontend.git |

本手冊以台南 61.218.209.214 環境部署為參考示例，實際部署請根據目標環境的 IP 地址及配置進行相應調整。